Relatório – AES-128 em dois ESP32 (ECB, UART2)

Alunos: Felipe Falcieri Macedo

Repositório: https://github.com/felipefalcieri-crypto/Criptografia-sim-trica.git

1. Implementação e validação

O AES-128 foi escrito do zero em aes128.h: SubBytes, ShiftRows, MixColumns, AddRoundKey, expansão de chave (10 rodadas) e as transformações inversas. Nenhuma biblioteca criptográfica faz parte do código entregue.

FIPS-197 Apêndice B (cifrar e decifrar): 3925841d02dc09fbdc118597196a0b32

FIPS-197 Apêndice C.1 (cifrar e decifrar): 69c4e0d86a7b0430d8cdb78070b4c55a

Exemplo do enunciado (2 blocos): OK

Três blocos ECB (LIGAR / DESLIGAR / LIGAR): OK

Duas placas ESP32 (UART2): texto original recuperado no receptor

Origem da S-box. Para cada byte calcula-se o inverso multiplicativo em GF(2^8), módulo $x^8+x^4+x^3+x+1$, e aplica-se uma transformação afim com constante 0x63. O inverso dá a não linearidade; a parte afim elimina pontos fixos. xtime multiplica por x: desloca 1 bit e, se o bit 7 "estourou", faz XOR com 0x1b.

2. Registros da Etapa 3 (hexadecimal)

Chave (ASCII 2023100120231002): 32303233313030313230323331303033
Mensagem com PKCS#7 (32 bytes):
413a32303233313030313b423a32303233313030320b0b0b0b0b0b0b0b0b0b0b
Texto cifrado final:
3a3784991ab0acaa88d5fd94cb7b74adcbea1d9e3e1f39da47111cf69064b166
Chave de rodada 0: 32303233313030313230323331303032
Chave de rodada 1: 373411f4060421c5343413f6050423c4

Cinco estados do bloco 1 (rodada 1)

Entrada (bloco 1 em claro): 413a32303233313030313b423a323032

1 - AddRoundKey inicial (chave 0): 730a000303030101020109710b020000

2 - SubBytes: 8f67637b7b7b7c7c777c01a32b776363

3 - ShiftRows: 8f7b01637b7c637b7777637c2b677ca3

4 - MixColumns: ea1953366a5d4c646840427520c24a3b

5 - AddRoundKey (chave 1): dd2d42c26c596da15c74518325c669ff

Após o AddRoundKey inicial, muitos bytes valem 00, 01, 02 ou 03: mensagem e chave são texto ASCII de dígitos e quase se cancelam no XOR. A S-box e a difusão da MixColumns desfazem essa regularidade já na primeira rodada.

Efeito avalanche (bloco 1)

1 bit da mensagem (byte 0, bit 0): 62 de 128 bits modificados (48,4 %)
1 bit da chave (byte 0, bit 0): 62 de 128 bits modificados (48,4 %)
Média sobre os 128 bits da mensagem: 63,3 de 128 (49,5 %)
Média sobre os 128 bits da chave: 63,4 de 128 (49,5 %)

O resultado confirma o esperado (~50 %): um único bit de entrada, seja da mensagem ou da chave, altera em média metade do bloco cifrado.

3. Transmissão pela UART2

Ligação TX→RX, RX→TX e GND (GPIO17 = TX2, GPIO16 = RX2), 9600 bps, 8N1. O quadro é enviado em bytes brutos: 0xAA | n | n × 16 bytes. No exemplo são 34 bytes (AA 02 + 2 blocos). O receptor descarta tudo até encontrar 0xAA, lê n (aceita de 1 a 8), lê exatamente 16·n bytes com tempo limite de 1 s, decifra, remove o PKCS#7 e exibe o texto no monitor serial. Nos testes com as duas placas, o transmissor enviou o cifrado 3a3784…74ad + cbea1d…b166 e o receptor exibiu A:20231001;B:20231002.

4. Por que não usar ECB?

No ECB cada bloco de 16 bytes é cifrado de forma independente e determinística (C = E(K, P)). Consequências:

1. **Blocos iguais geram cifrados iguais.** Nos três blocos de teste, "LIGAR LED VERDE." aparece duas vezes e produz e0c22a6f02664f487dc9528642203dec nas duas, enquanto "DESLIGAR LED" produz 2b4d065ea65478b60696616244cd5070. Um espião na UART, sem conhecer a chave, vê que os blocos 1 e 3 são iguais e que o 2 é diferente. Se observar o efeito no sistema (o LED acendendo), associa cada bloco cifrado ao seu comando e passa a reconhecê-lo.
2. **Troca, remoção e repetição de blocos passam despercebidas.** Como um bloco não depende dos vizinhos nem da posição, o atacante pode reordenar, apagar ou duplicar blocos, ou reenviar um quadro antigo inteiro (replay), e o receptor decifra tudo normalmente. O PKCS#7 só valida o padding do último bloco: na simulação, um bit alterado no bloco 1 não gerou erro.
3. **Não há IV nem aleatoriedade.** A mesma mensagem com a mesma chave sempre gera o mesmo texto cifrado, o que permite reconhecer mensagens repetidas.

O que usar na prática:

- **CBC:** cada bloco é combinado (XOR) com o cifrado anterior e o primeiro com um IV novo e imprevisível a cada mensagem; blocos iguais deixam de gerar cifrados iguais.
- **CTR:** transforma a cifra de bloco em cifra de fluxo, com um nonce/IV que nunca se repete para a mesma chave.
- **GCM (preferível):** CTR mais um tag de autenticação, que detecta troca, remoção, repetição ou alteração de qualquer bit. CBC e CTR sozinhos não autenticam e exigiriam um MAC separado.

O IV viaja junto da mensagem, em claro; a chave permanece secreta. Contra replay, ainda seria necessário um contador ou número de sequência autenticado.